Reading 6.3: Model-free Learning – Monte Carlo and Temporal Difference Learning

In this lesson we will explore two key approaches in Reinforcement Learning: Model-Based and ModelFree methods. By the end of this lesson, you will:

1. Understand the differences between Model-Based and Model-Free learning
2. Uncover Monte Carlo and Temporal Difference learning
3. Understand the importance of exploration in RL

Model-Based Learning: Dynamic Programming

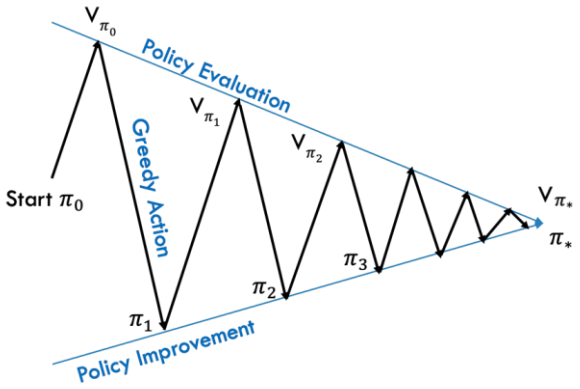
Dynamic Programming (DP) operates in scenarios where the agent has full visibility of the environment, enabling it to compute the optimal policy without needing direct interaction with the environment. However, in most real-world RL problems, the agent lacks knowledge about the inner workings of the environment, rendering DP impractical for most applications. As a consequence, Dynamic Programming will not be covered in this module.

Prediction and Control Problem

In traditional RL problem solving, we recognise 2 stages:

- Policy Evaluation or the Prediction Problem
  We compute $\boldsymbol{V}_{\pi}$(s) or $\boldsymbol{Q}_{\pi}$(s,a) for an arbitrary MDP with a given policy $\boldsymbol{\pi}$
- Policy Iteration or the Control Problem

Once we calculated the state values, we find the optimal policy $\boldsymbol{\pi}_*$ by acting greedily after each policy evaluation step. Greedy actions are actions that increase the state value. We repeat these steps until convergence: $\boldsymbol{\pi_0 \rightarrow \pi_1 \rightarrow \pi_2 \rightarrow \cdots \rightarrow \pi_*}$

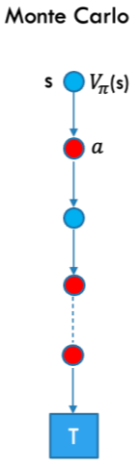


Model-Free Learning

Model-free learning refers to methods where the agent learns optimal behaviour while interacting directly with the environment, using trial and error to estimate the values of actions or states.

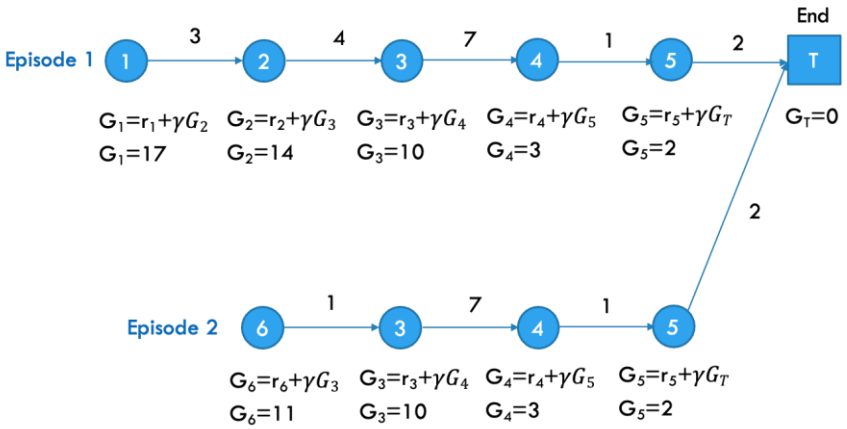
Monte Carlo: Prediction Problem

In policy evaluation we aim to calculate V_π(s) for all states. In MC we sample experience through interactions with the environment. The experience is represented by a set of states, actions and rewards, called an episode. For MC to work, each episode needs to end in a terminal state. In MC there is no bootstrapping!



Monte Carlo: Control Problem

Let's look at 2 different episodes. As usual we set γ=1. Let's calculate the returns starting from the terminal state and moving inwards. Per definition the terminal state has a return of zero. The other returns are calculated using Bellman: $G_t = r' + \gamma G_{t+1}$



Episode 1	Average
G_1=17	17
G_2=14	14
G_3=10	10
G_4=3	3
G_5=2	2
G_T=0	0

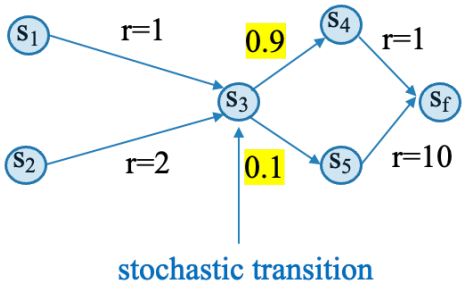
Episode 1	Episode 2	Average
G_1=17	G_1=0	8.5
G_2=14	G_2=0	7
G_3=10	G_3=10	10
G_4=3	G_4=3	3
G_5=2	G_5=2	2
G_6=0	G_6=11	5.5
G_T=0	G_T=0	0

In MC we generate a series of episodes that terminate. We subsequently add up and average the returns in the specific states. If we repeat the experiment many times, the average return for each state will converge to the expected return E[G] and E[G] is nothing more than the definition for the state value $V_\pi(s)$.

Temporal Difference Learning (TD)

TD learning is a mix of Dynamic Programming and Monte Carlo. TD learning is model-free and learns directly from experience. In TD learning we aim to learn from incomplete episodes by bootstrapping. Let's derive the formula for TD using a simple example:

We have 6 states of which s_3 triggers a stochastic transition: in 90% of the cases the agent transitions to s_4 while the remaining 10% of the time the agent transitions to s_5. The value in each state is given by an immediate reward r' plus $\gamma.V_\pi(s')$. In the end state sf the value is zero. A transition starting in s_3 does not yield a reward. Let's calculate our state values using MC. We start in the terminal state and work our way backwards:



$$V_\pi(s) = \begin{cases} 0 & \text{if s is end state} \\ E[r' + \gamma V_\pi(s')] & \text{in all other cases} \end{cases}$$

Bellman 1

$V(s_f) = 0$
$V(s_4) = +1 + 0 = 1$
$V(s_5) = +10 + 0 = 10$
$V(s_3) = 0.9*(0+1)+0.1(0+10)=1.9$
$V(s_2) = 2+1.9 = 3.9$
$V(s_1) = +1+1.9 = 2.9$

In MC, we sample for example 4 full episodes from this data. We have calculated that $V_{(s_1)} = 2.9$ using MC. Let's estimate $V_{(s_1)}$ after 3 and 4 episodes. Via our episode table we can calculate:

Ep 1	s_1	+1	s_3	0	s_4	+1	s_f	+2
Ep 2	s_1	+1	s_3	0	s_5	+10	s_f	+11
Ep 3	s_1	+1	s_3	0	s_4	+1	s_f	+2
Ep 4	s_1	+1	s_3	0	s_4	+1	s_f	+2

$$V(s_1)_3 = \frac{2+11+2}{3} = \frac{15}{3} = 5 \quad (1)$$
$$V(s_1)_4 = \frac{15+2}{4} = \frac{17}{4} = 4.25$$

We are particularly interested in estimating the value of s_1 after 4 episodes (t+1) based on the value of s_1 after 3 episodes (t). Looking at equation (1) we learn that we can calculate $V(s_1)_4$ by:

$$V(s_1)_4 = \frac{3*5+2}{4} = \frac{17}{4} = 4.25$$

$r(s_1)_4$, $V(s_1)_3$, 3 episodes

$$V(s_1)_t = \frac{(t-1)*V_{t-1}(s_1)+r_t(s_1)}{t}$$

$$V(s_1)_t = \frac{(t-1)}{t}V_{t-1}(s_1) + \frac{r_t(s_1)}{t}$$

$$V(s_1)_t = V_{t-1}(s_1) - \frac{1}{t} * V_{t-1}(s_1) + \frac{r_t(s_1)}{t}$$

$$\alpha_t = \frac{1}{t}$$

$$V(s_1)_t = V_{t-1}(s_1) + \alpha_t[r_t(s_1) - V_{t-1}(s_1)]$$

Temporal Difference

The temporal difference is the difference between the reward that we get now r' and the value estimate $V_\pi(s)$ that we had in previous time stamp. Our update rule becomes:

TD Target

$$V_\pi(s_t) \leftarrow V_\pi(s_t) + \alpha[r_{t+1} + \gamma V_{t+1} - V_\pi(s_t)]$$

TD Error

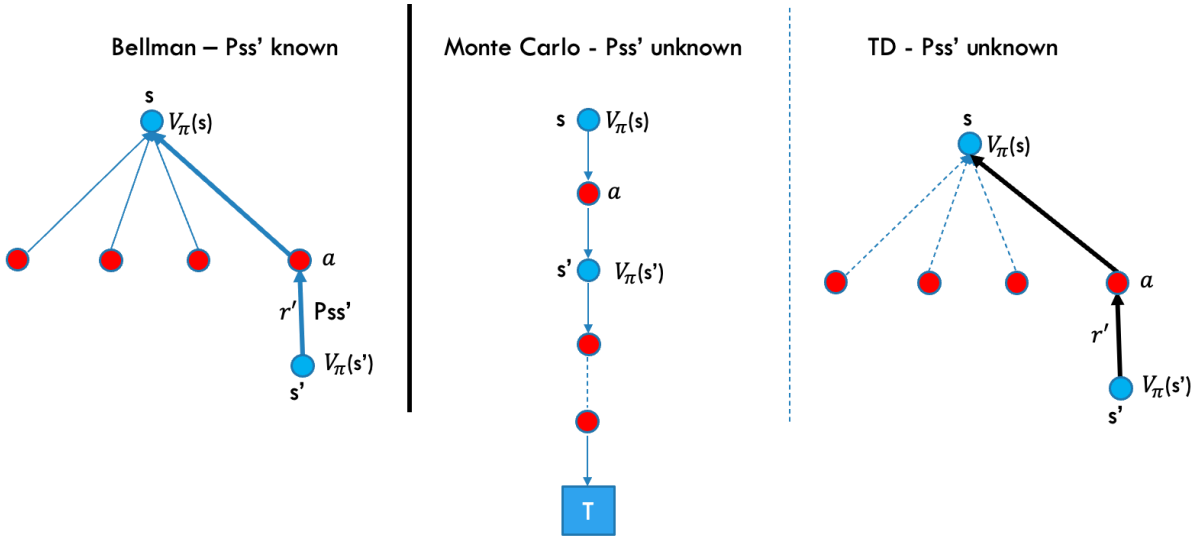
update

S, A, R, S, A, S, A, R, S, A,....

t+1

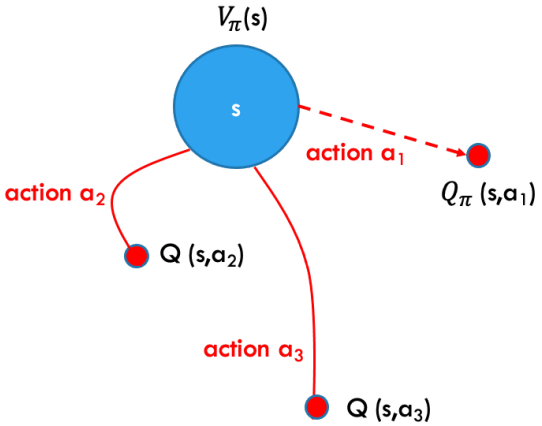
In TD, our agent takes an action resulting in an immediate reward r_{t+1} and a new state s_{t+1}. Only after having landed in the new state s_{t+1} we make an update to the previous state value. Both $V_{(s_{t+1})}$ and $V(s_t)$ are estimates and in TD we learn an estimate from an estimate.

We are now able to show the backup diagrams for all algorithms:



Exploration vs. Exploitation

Let us consider following situation and assume a deterministic policy π. The deterministic policy prescribes to take the action a_1. As a result, the agent will never be able to explore the returns from the other actions a_2 and a_3 while they might be better than a_1. This exploration problem hinders the learning process.



One option to deal with this trade-off is to opt for an ε-greedy policy. These policies are stochastic policies and usually take the greedy action but occasionally take a random action. After running enough episodes it will have taken every action at least once in every state as the agent will follow a slightly different policy in each episode.

Try it Yourself:

Revisit the Ski Resort scenario introduced when we discussed the Markov Decision Process (MDP). Apply TD to determine the state values of all Areas, including the hotel. Reflect on the results below and make sure that you understand how we get to the state values.

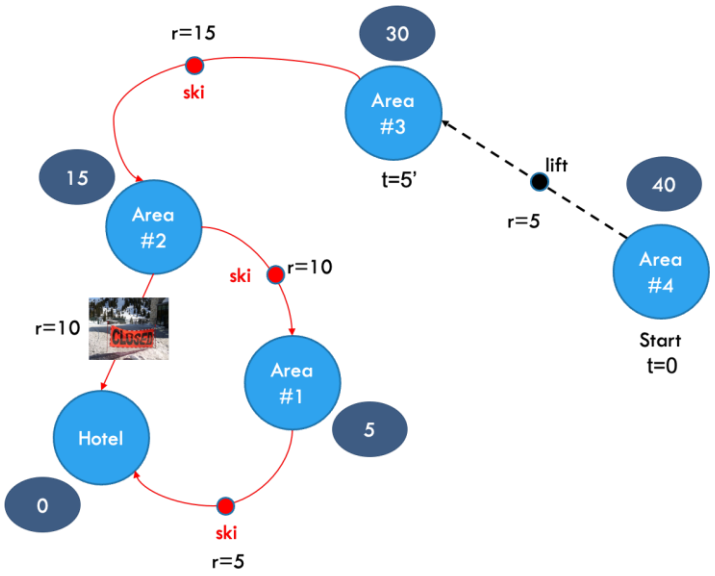
The time it takes to get from $Area_4$ back to the Hotel under normal circumstances is 30'. Assume a scenario where you ski from $Area_4$ back to the Hotel in the late afternoon. Today the slope connecting $Area_2$ and the Hotel is closed.

We assume that $\alpha = \gamma = 1$. In TD, our update is given by the following update rule:

$$V(S_t) \leftarrow V(S_t) + \alpha[r_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

$$V(S_t) \leftarrow V(S_t) + [r_{t+1} + V(S_{t+1}) - V(S_t)]$$

$$V(S_t) \leftarrow r_{t+1} + \gamma V(S_{t+1})$$



$Area_4$ $V(Area_4) \leftarrow 5 + V(Area_3)$

40

$V(Area_4) \leftarrow 5 + 35$

$Area_3$ $V(Area_3) \leftarrow 15 + V(Area_2)$

30

$V(Area_3) \leftarrow 15 + 15$

$Area_2$ $V(Area_2) \leftarrow 10 + V(Area_1)$

15

$V(Area_2) \leftarrow 10 + 5$

$Area_1$ $V(Area_1) \leftarrow 5 + V(Hotel)$

5

$V(Area_1) \leftarrow 5 + 0$

Taking the lift from $Area_4$ to $Area_3$ results in a reward of 5. The estimated (guessed) value of $Area_3$ given an unplanned sanitary stop is 35. As a result, we update the value for $Area_4$ to 40.

Skiing from $Area_3$ to $Area_2$ results in a reward of 15 while the estimated value of $Area_2$ is 15, hence we update the value of $Area_3$ to 30. Skiing from $Area_2$ to $Area_1$ results in a reward of 10 while the estimated value of $Area_1$ is 5, hence we update the value of $Area_2$ to 15. Similarly, we update the value for $Area_1$ to 5.